

## unique

- $O(N)$  去重 (只去相邻重复)
- 1. 离散化 (Sort + Unique + Erase) 必须先 Sort!  $m = \text{unique}(a, a+n) - a$ ; 返回去重后末尾指针。

## next\_permutation $O(N)$

- 下一个全排列
- 暴力枚举全排列
- 生成字典序
- `do { ... } while(next_permutation(all(s)));`

## lower\_bound $O(\log N)$

- 找  $\geq val$  的第一个位置
- LIS (最长上升子序列)
- 查找排名/前驱后继返回迭代器。
- `idx = lower_bound(all(v), val) - v.begin();`

## \_\_gcd

- $O(\log V)$
- 最大公约数
- 数论、化简分数
- `lcm(a,b) = a/gcd(a,b)*b`

## fill

- $O(N)$
- 赋值
- 初始化数组/容器
- `fill(a, a+n, val);` 比 `memset` 安全, 支持任意值 (`memset` 只能 0/-1/0x3f)。

## vector 我的最爱

- 支持可变大小, 不支持开出来的大小清空
- $O(1)$  `push_back()`, `pop_back()`
- $O(n)$  `insert()`, `erase()`
- 连续空间, 可以直接指针++
- `resize()` 改变大小且填充默认值; `reserve()` 只预留空间不改大小 (优化常数神器)。
- 邻接表存图 `vector G[N]`

## deque 支持双向

- 头尾插入放出  $O(1)$  `push_front`; `push_back`; `pop_front`; `pop_back`
- 适用于滑动窗口
- 0-1 BFS (边权只为0或1的最短路)

## string

- 拼接 + 是  $O(N)$
- 配合 stringstream 做类型转换
- s.find() 找不到返回 string::npos。
- 千万别在循环里  $s = s + c$ , 要用  $s += c$ 。

## priority\_queue 优先队列

- push, pop,  $O(\log n)$  方便贪心最大最小
- 可以通过重载运算符来达成最大或者这最小

## list

- $O(1)$ 删除任意节点 **erase**
- 不能随机访问

## stack

- 后缀表达式
- DFS

## queue

- BFS
- **注意**队列不能遍历, 没有 operator[], 如果要遍历却不出队可以使用 deque

## set 自动去重的有序集合

- $O(\log n)$ 查找 **find()**; 插入于删除
- **支持****lower\_bound(x)** ``upper\_bound(x) 依旧  $O(\log n)$
- 场景: 动态维护有序序列且快速去重; 坐标压缩
- 清空数组 set.clear()
- 注意: **bc.push\_back({t, (int)cnt.size()})** (这里的 cnt 是 set 类型的) 类似于这一句需要强制类型转换, 因为 set.size() 函数返回的是无符号类型

## multiset 和 set 完全一样但是允许重复元素

- 动态维护中位数 场景: 滑动窗口中位数
- 贪心 (类似堆, 但支持删任意值) 实现  $O(\log n)$  美丽复杂度的查找与删除, 利于维护优先队列 (包括 **lower\_bound(x)** ``upper\_bound(x))
- **大坑: erase(val) 会删掉所有等于 val 的元素; erase(iter) 只删一个。** iter 是迭代器

## map

(例子定义: map<int,int> mp;)

- 增删查均为  $O(\log N)$
- 离散化
- 自动排序的去重统计

- 启发式合并
- 支持 mp[x]快速访问与插入
- mp.count(key)判断成员是否存在（**仅仅判断请用这个**，如果用operator会插入对应键值对，默认数值为0）
- 离散化映射；建图
- mp的遍历: for (auto it = nums.begin(); it != nums.end(); it++) 提取内容写法: auto hsh=it->second;

std::unordered\_set / unordered\_map —— 哈希表（平均  $O(1)$ ）但是好像最坏能到 $O(n)$

我不会，也没做过类似的题，以下内容均由ai生成

- 平均时间复杂度 $O(1)$ ，find, insert, 等操作都快
- 无序
- 不支持lower\_bound
- 键值唯一
- 哈希冲突严重时退化成链表 $O(n)$
- 字符串/大数 快速映射
- 频率统计

堆 std::priority\_queue

- 好东西
- 最短路dijkstra
- 贪心
- 合并果子
- 写法:
- 在 priority\_queue 的 cmp 中，return true 的含义是：“左边这个元素(a) 比 右边这个元素(b) 优先级更低。”
- top  $O(1)$
- push/pop  $O(\log N)$

```
struct Node {
    int val;
    int id;
};

// === 核心模板：自定义比较结构体 ===
struct cmp {
    // 传入 const引用，速度快且安全
    // 返回 true 表示: a 的优先级比 b "低" (a 应该沉在下面)
    bool operator()(const Node& a, const Node& b) {

        // --- 场景 1: 我是小根堆 (数值小的在堆顶) ---
        // 逻辑: 如果 a 的值 大于 b, 说明 a 比较"重", a 应该沉下去
        return a.val > b.val;

        // --- 场景 2: 我是大根堆 (数值大的在堆顶) ---
    }
};
```

```

        // 逻辑: 如果 a 的值 小于 b, 说明 a 比较“弱”, a 应该沉下去
        // return a.val < b.val;

        // --- 场景 3: 多关键字排序 ---
        // 比如: 先按 val 小的排, val 一样按 id 大的排
        // if (a.val != b.val) return a.val > b.val; // val大的沉下去(小根)
        // return a.id < b.id; // id小的沉下去(大根)
    }
};
// === 核心模板:比较数字 ===

int dist[10005]; // 全局距离数组

// 堆里只存节点编号 int
struct cmp {
    bool operator()(int u, int v) {
        // 哪怕堆里只有 int, 我也可以引用外部数组来比较
        // 我要 distance 小的在堆顶 (Dijkstra 常见写法)
        return dist[u] > dist[v];
    }
};

// 使用
priority_queue<int, vector<int>, cmp> pq;

int main() {
    // 声明方式: <类型, 容器, 比较器>
    // 这里的 vector<Node> 是堆的底层容器, 必写, 但不用管它
    priority_queue<Node, vector<Node>, cmp> pq;

    pq.push({10, 1});
    pq.push({5, 2}); // 5 最小, 应该在最上面
    pq.push({20, 3});
    //用push是因为这样子两个都会变成node类型
    while(!pq.empty()) {
        Node top = pq.top();
        pq.pop();
        cout << "Val: " << top.val << endl; // 输出顺序: 5 -> 10 -> 20
    }
    return 0;
}

```